

# Low-Power VLSI Design of LDPC Decoder Using DVFS for AWGN Channels

Wei Huang Wang, Gwan Choi

Department of Electrical and Computer Engineering  
Texas A&M University, TX 77840  
Email: whwang@tamu.edu, gchoi@ece.tamu.edu

Kiran K. Gunnam

Channel Architecture, Storage Peripherals Group,  
LSI Corporation, Milpitas, CA 95035  
Email: kiran.gunnam@lsi.com

**Abstract**—This paper presents a low-power LDPC decoder design for additive white Gaussian noise (AWGN) channels. The proposed decoding scheme provides constant-time decoding and thus facilitates real-time applications where guaranteed data rate is required. It analyzes each received data frame to estimate the maximum number of necessary iterations for frame convergence. The results are then used to dynamically adjust decoder frequency and switch between multiple-voltage levels; thereby energy use is minimized. It differs from recent publications on speculative LDPC decoding for block-fading channels. Our approach addresses the more difficult problem of decoding requirement prediction for data frames in AWGN channels. It is also directly applicable for fading channels. A decoder architecture utilizing offset min-sum layered decoding algorithm is presented. Up to 30% saving in decoding energy consumption is achieved with negligible coding performance degradation.

## I. INTRODUCTION

Low-density parity-check (LDPC) codes are special cases of error correcting codes originally proposed by Gallager [1] in 1960's and rediscovered in late 1990's [2]. LDPC codes have recently gained a significant attention because of their near Shannon-limit performance and high throughput. They have been successfully adopted in next-generation standards, such as *IEEE802.11n*, *IEEE802.16e*, DVB-S2, etc.

LDPC codes are defined by a sparse parity check matrix  $H = [H_{mn}]$  that consists mostly of 0's, as an example shown in Figure 1. The  $H$  matrix of  $(d_c, d_v)$  regular LDPC code has the following properties. Each column contains a small fixed number  $d_v$  of 1's and each row contains a small fixed number  $d_c > d_v$  of 1's. The block length of this code is  $n$  which is equal to the number of columns in the  $H$  matrix. Suppose that the number of data bits before the channel encoding is  $l$ , then the number of rows of this  $H$  matrix is  $m = n - l$ . Rate of this code is defined as  $l/n = 1 - d_v/d_c$ . The code words consist of all one-dimensional row vectors that span the null space of the parity check  $H$  matrix. The number for  $d_v$  and  $d_c$  should be no less than 3 and 6 respectively, for good coding performance.

LDPC codes can be equivalently represented in a bipartite graph, illustrated by Figure 2. The widely used message-passing algorithms exchange information between the variable-nodes and check-nodes in an iterative fashion. In practice, the LDPC decoder is typically set to run for data convergence until a prescribed maximum number of iterations

$$H = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \end{bmatrix} \begin{matrix} S_1 \\ S_2 \\ S_3 \\ S_4 \\ S_5 \\ S_6 \end{matrix}$$

Check nodes

$V_1 \ V_2 \ V_3 \ V_4 \ V_5 \ V_6 \ V_7 \ V_8 \ V_9$

Fig. 1. An example  $H$  matrix of  $d_c = 3$  and  $d_v = 2$ .

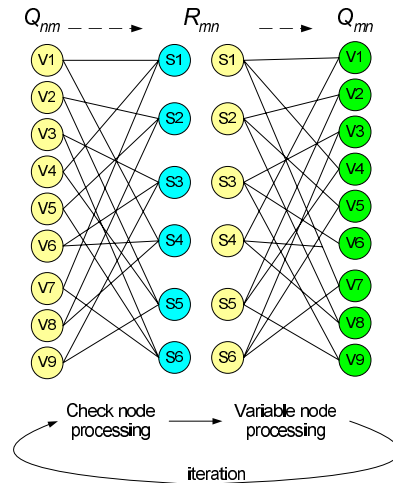


Fig. 2. Bipartite graph representation and two phase message passing decoding algorithm.

(e.g. 20) depending on the code rate [3]–[6]. However, the actual number of decoding iterations varies from frame to frame. In the case that channel data comes in constant-time interval, a conventional decoder has to be configured to accommodate the worst case scenario. As a result, the decoder often remains idle since for most frames, the decoding process ends far earlier than the maximum number of iterations. Thus it is not power efficient. In the decoders proposed in [7]–[10] based on an on-the-fly computation paradigm, optimized dataflow graphs are introduced to reduce the logic and internal memory requirements of the LDPC decoder and at the same time the decoder's parallelization is tailored to average number

of decoding iterations for the target frame error rate for the operating SNR region. These decoders buffer statistically for different parallelization based on average number of decoding iterations while ensuring the performance similar to that of a fixed iteration decoder configured for maximum number of iterations [7]–[10]. These decoders are almost fully utilized and runs at the maximum frequency. The proposed paper improves the system efficiency in a different approach by adaptively adjusting the frequency and voltage of the decoder to meet the required iterations for each incoming frame.

There have been researches on early termination of frame that can not be decoded even if the maximum iterations are applied [5], [6]. In both of papers, early termination of the iterative process is determined by checking the messages during the decoding. Their attempts are to dynamically switch off the hardware when no additional iteration will amount to improvement in decoding performance. Such architectures yield unpredictable frame-completion time which makes interfacing with the application modules rather difficult. Wang, *et al.* address the problem of dynamic voltage and frequency scaling (DVFS) for block fading channels in [11]. The presented approach predicts the required decoding effort by evaluating number of checks in error for each frame. Such approach will not work for AWGN channels where correlation between initial check error and number of decoding iterations is insufficient. This paper describes a more comprehensive design that accommodate both fading channels and the more difficult case of AWGN channels.

The rest of the paper is organized as following: Section II presents the LDPC decoding algorithm based on offset min-sum and layered decoding. The low-power adaptive decoder architecture is proposed in Section III. Implementation results as well as discussion is given in Section IV. Section V concludes the paper.

## II. LDPC LAYERED DECODING BASED ON OFFSET MIN-SUM ALGORITHM

The array LDPC codes [12] are specified by three parameters: a prime number  $p$  and two integers  $k$  and  $j$  such that  $j < p$  and  $k < p$ . It is given by Equation (1):

$$H = \begin{bmatrix} I & I & I & \cdots & I \\ I & \alpha & \alpha^2 & \cdots & \alpha^{k-1} \\ I & \alpha^2 & \alpha^4 & \cdots & \alpha^{2(k-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ I & \alpha^{j-1} & \alpha^{(j-1)2} & \cdots & \alpha^{(j-1)(k-1)} \end{bmatrix} \quad (1)$$

where  $I$  is the  $p \times p$  identity matrix, and  $\alpha$  is a  $p \times p$  permutation matrix representing a single left or right cyclic shift of  $I$ . Power of  $\alpha$  in  $H$  denotes multiple cyclic shifts, with the number of shifts given by the value of the exponent.

Assume binary phase shift keying (BPSK) modulation (a 1 is mapped to  $-1$  and a 0 is mapped to 1) over an additive white Gaussian noise (AWGN) channel. The received values  $y_n$  are Gaussian with mean  $x_n = 1$  and variance  $\delta^2$ . The iterative two-phase message-passing (TPMP) algorithm, also

known as belief-propagation (BP) algorithm [13], [14] is computed in two phases. One is a check node processing and the other is variable node processing. In the check node processing, each row of the parity matrix is checked to verify that parity check constraints are satisfied. In the variable node processing the probability will be updated by summing up the other probabilities from the rest of the rows and the a priori probabilities from the channel output. The message-passing algorithm can be simplified to the belief-propagation based algorithm (also called Min-Sum algorithm) [15]. While greatly reducing the decoding complexity in implementation, the Min-Sum degrades the coding performance. The improved BP based algorithm, Normalized-Min-Sum and Offset-Min-Sum eliminates this performance degradation.

Following the same notation in [8], the check node processing can be expressed as:

$$R_{mn}^{(i)} = \delta_{mn}^{(i)} \max(\kappa_{mn}^{(i)} - \beta, 0), \quad (2)$$

$$\kappa_{mn}^{(i)} = |R_{mn}^{(i)}| = \min_{n' \in N(m) \setminus n} |Q_{n'm}^{(i-1)}|, \quad (3)$$

where  $Q_{nm}^{(i)}$  is the message from variable node  $n$  to check node  $m$ ,  $R_{mn}^{(i)}$  is the message from check node  $m$  to variable node  $n$ , and superscript  $i$  denotes the  $i^{th}$  iteration.  $M(n)$  is the set of the neighboring check nodes for variable node  $n$ , and  $N(m)$  is the set of the neighboring variable nodes for check node  $m$ ,  $\beta$  is a positive constant and depends on the code parameters [15]. The sign of check-node message  $R_{mn}^{(i)}$  is defined as

$$\delta_{mn}^{(i)} = \left( \prod_{n' \in N(m) \setminus n} \text{sgn}(Q_{n'm}^{(i-1)}) \right). \quad (4)$$

In the variable node processing,

$$Q_n = L_n^{(0)} + \sum_{m \in M(n) \setminus m} R_{mn}^{(i)} \quad (5)$$

where the log-likelihood ratio of bit  $n$  is  $L_n^{(0)} = y_n$ . For final decoding

$$P_n = L_n^{(0)} + \sum_{m \in M(n)} R_{mn}^{(i)} \quad (6)$$

A hard decision is taken by setting  $\hat{x}_n = 0$  if  $P_n(x_n) \geq 0$ , and  $\hat{x}_n = 1$  if  $P_n(x_n) \leq 0$ . If  $\hat{x}H^T = 0$ , the decoding process is finished with  $\hat{x}_n$  as the decoder output; otherwise, repeat processing of Equation (2)–Equation (6). If the decoding process does not end within predefined maximum number of iterations,  $it_{max}$ , stop and output an error message flag and proceed to the decoding of the next data frame.

Mansour, *et al.* introduces the concept of turbo decoding message passing (also called as layered decoding) for their AA-LDPC codes using BCJR [4], which is able to reduce the number of iterations required by up to 50% without performance degradation when compared to the standard message passing algorithm. Contrast to two phase message passing

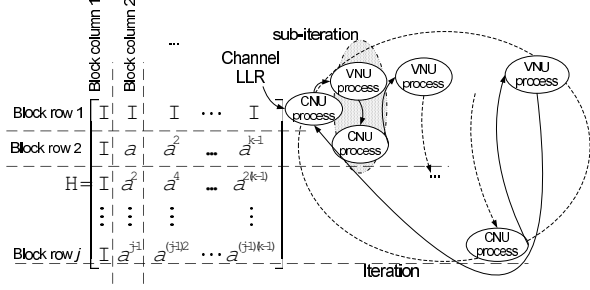


Fig. 3. In layered decoding, the  $H$  matrix is viewed as concatenation of  $d_v$  sub-codes and each block row (layer) is updated individually.

algorithm, where all check-nodes are updated simultaneously in each iteration, layered decoding view the  $H$  matrix as a concatenation of  $j = d_v$  sub-codes, as shown in Figure 3. The  $H$  matrix is divided into different block-rows and block-columns. After the check-node processing of one layer, the updated messages are immediately used to calculate the variable-node message, whose results are then applied to next layer of sub-code. Each iteration in the layered decoding algorithm is composed of  $j$  sub-iterations. The processing of one block-row is called a sub-iteration and each iteration is composed of  $j = d_v$  sub-iterations. Mathematically, the layered decoding algorithm can be described as in Algorithm 1:

```

 $\tilde{R}_{l,n}^{(0)} = 0, \forall l \in [1, d_v], n \in [1, d_c]$ 
 $\tilde{P}_n = \tilde{L}_n^{(0)}, \forall n \in [1, d_c]$ 
foreach  $i = 1, 2, \dots, i_{tmax}$  do
  foreach  $l = 1, 2, \dots, d_v$  do
    foreach  $n = 1, 2, \dots, d_c$  do
       $[\tilde{Q}_{l,n}^{(i)}]^{S(l,n)} = [\tilde{P}_n]^{S(l,n)} - \tilde{R}_{l,n}^{(i-1)}$  (7)
       $\tilde{R}_{l,n}^{(i)} = f\left([\tilde{Q}_{l,n'}^{(i)}]^{S(l,n')}\right), \forall n' \in [1, d_c]$  (8)
       $[\tilde{P}_n]^{S(l,n)} = [\tilde{Q}_{l,n}^{(i)}]^{S(l,n)} + \tilde{R}_{l,n}^{(i)}$  (9)

```

Algorithm 1: Layered decoding for array LDPC.

### III. LOW-POWER ADAPTIVE LDPC DECODER ARCHITECTURE

This section presents the adaptive decoder architecture for additive white Gaussian noise (AWGN) channel. Decoding effort is estimated before the decoding process of every frame in block-fading channels [11], [16]. Decoding energy is saved via truncating the distribution of decoding effort and applying dynamic voltage and frequency scaling (DVFS) technique. This approach, however, does not work for LDPC decoding in AWGN channels.

While number of check errors for the coming channel data still partially represents the degree of noise impairment, and indicate effort required for decoding, the correlation between number of check errors and number of decoding iteration though, becomes much less. However, this correlation can be

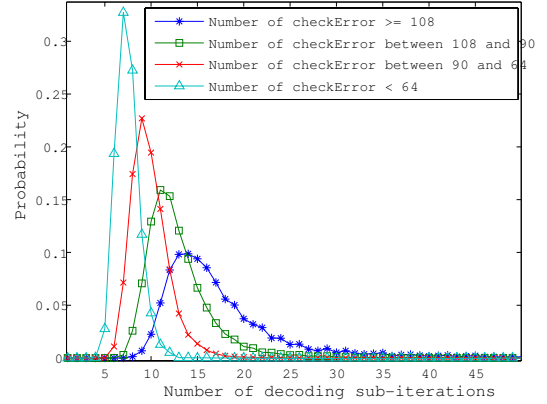


Fig. 4. Distribution of decoding sub-iteration for different number of check errors after three sub-iterations with channel  $E_b/N_0$  at 3.7dB.

```

 $\tilde{R}_{l,n}^{(0)} = 0, \forall l \in [1, d_v], n \in [1, d_c]$ 
 $\tilde{P}_n = \tilde{L}_n^{(0)}, \forall n \in [1, d_c]$ 
foreach  $l = 1, 2, 3$  do
  foreach  $n = 1, 2, \dots, d_c$  do
     $[\tilde{Q}_{l,n}^{(1)}]^{S(l,n)} = [\tilde{P}_n]^{S(l,n)}$ 
     $\tilde{R}_{l,n}^{(1)} = f\left([\tilde{Q}_{l,n'}^{(1)}]^{S(l,n')}\right), \forall n' \in [1, d_c]$ 
     $[\tilde{P}_n]^{S(l,n)} = [\tilde{Q}_{l,n}^{(1)}]^{S(l,n)} + \tilde{R}_{l,n}^{(1)}$ 
   $\hat{x}_n = \text{hard decision of } P_n$ 
  if  $\hat{x}H^T \in (cErrThre_1, +\infty)$  then  $i_{sub,max} = C_1$ 
  else if  $\hat{x}H^T \in (cErrThre_2, cErrThre_1)$  then
     $i_{sub,max} = C_2$ 
  else if  $\hat{x}H^T \in (cErrThre_3, cErrThre_2)$  then
     $i_{sub,max} = C_3$ 
  else if  $\hat{x}H^T \in (0, cErrThre_3)$  then
     $i_{sub,max} = C_4$ 
  foreach  $i_{sub} = 4, \dots, i_{sub,max}$  do
     $i = \lceil i_{sub}/d_v \rceil$ 
     $l = (i_{sub} - 1) \bmod (d_v) + 1$ 
    foreach  $n = 1, 2, \dots, d_c$  do
       $[\tilde{Q}_{l,n}^{(i)}]^{S(l,n)} = [\tilde{P}_n]^{S(l,n)} - \tilde{R}_{l,n}^{(i-1)}$ 
       $\tilde{R}_{l,n}^{(i)} = f\left([\tilde{Q}_{l,n'}^{(i)}]^{S(l,n')}\right), \forall n' \in [1, d_c]$ 
       $[\tilde{P}_n]^{S(l,n)} = [\tilde{Q}_{l,n}^{(i)}]^{S(l,n)} + \tilde{R}_{l,n}^{(i)}$ 

```

Algorithm 2: Adaptive LDPC decoding for AWGN channel.

recovered after several initial decoding iterations. Analysis in this section has been carried out based on a sample array LDPC code, with parameters  $d_v = 5$ ,  $d_c = 25$ , and  $p = 67$ . And layered decoding based on offset min-sum (OMS) algorithm is used for decoding. Similar analysis can be extended to other array LDPC codes. It is shown in Figure 4 that after three sub-iterations, the number of check errors remaining is

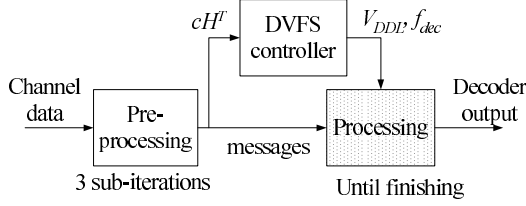


Fig. 5. Block diagram of the proposed adaptive decoding for AWGN channel.

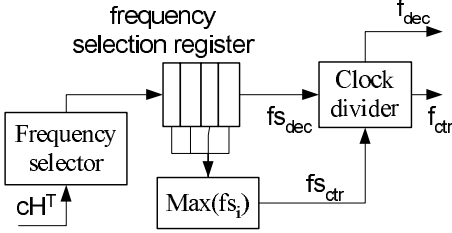


Fig. 6. DVFS controller for adaptive LDPC decoding.

correlated with the total number of decoding iterations. As such, the adaptive decoding for AWGN channel is proposed as in Algorithm 2.

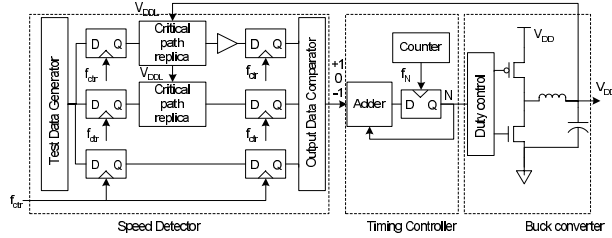


Fig. 7. Variable supply voltage scheme [17].

There are two sets of parameters that need to be determined experimentally for each specific application: the check-error threshold values  $cErrThre_{[1:3]}$  and the maximum number of decoding sub-iterations  $C_{[1:4]}$ . Note that this algorithm can be easily extended to fading channels as well.

An efficient decoder architecture is proposed based on the layered decoding architecture [7]. Block diagram of the decoder is shown in Figure 5. There are three main blocks: the pre-processing unit, the processing unit and the DVFS controller. For each incoming data frame, the first three decoding sub-iterations take place in the pre-processing unit. The check-node messages and  $P$  sum messages are stored in frame buffers and the value of  $cH^T$  will be sent to the DVFS controller for voltage and frequency adjustment. The processing unit takes over the unfinished decoding process by reading messages from the frame buffers. It operates in the variable voltage  $V_{DDL}$  domain. Decoding power is reduced since majority of the decoding process is done with the processing unit.

Diagram of the DVFS controller is presented in Figure 6. Number of check errors is calculated after three sub-iterations of each frame. Since  $cH^T$  is needed by the decoder for decoding termination decision and it can be reused, this calculation does not impose any additional hardware resource. Because

level of the voltage supply can not be changed instantly, frame buffer is required between the pre-processing unit and the processing unit, the frequency-selection register stores decoding iteration information for the corresponding data frames. The clock divider divides the fast system clock into slower clock signals according to the frequency selection register. Clock divider is preferred over other designs such as phase-loop locker (PLL) in [18], because it provides reasonable frequency resolution for the decoding policy and capability to change immediately.  $f_{dec}$  clocks the decoder for current frame, and  $f_{ctr}$  is sent to the variable voltage scheme, as shown in 7.  $f_{ctr}$  is conservatively generated as the fastest clock such that the voltage supply will be within safe region for operation. The variable voltage scheme is studied in depth by T. Kuroda, *et al.*, and the readers are referred to [17] for the details.

Figure 8 shows the adaptive LDPC decoder architecture in detail. The CNUs in the figure take the  $d_c$  variable-node message associated with each check-node serially, and computes the compressed check-node message in the form of  $M1$ ,  $-M1$ ,  $M2$  or  $-M2$ , and index of  $M1$  [7]–[9], in which  $M1$  is the least magnitude of all  $d_c$  variable-node messages and  $M2$  is the second least magnitude. These compressed check-node messages are called Final States (FS) and they are stored in FS buffers. Check-node messages to each associated variable-node are sent out serially again and they are selected from  $M1$ ,  $-M1$  and  $M2$  or  $-M2$  by index and sign comparison, where the signs of  $R$  messages are stored in sign FIFO. This approach reduces check-node message memory requirement by 50% to 80% depending on the code parameters. As soon as check-node message to the first connected variable-node are ready, the corresponding  $P$  sum message can be computed. Therefore, the  $Q$  message is ready for check-node message processing of next sub-iteration. As such, each CNU operates on two layers of the  $H$  matrix simultaneously: selecting check-node message for one layer and computing FS for the next layer.

The pre-processing unit operates in the nominal  $V_{DD}$  voltage domain. In the first iteration,  $Q^{shift}$  equals channel LLR or  $P$  message subtracting  $R_{old}$ , in which  $R_{old}$  is check node message from last iteration, i.e.  $R_{l,n}^{i-1}$  in Equation (7). Since both FS register and sign FIFO are reset to zero when decoding starts, and remains zero during the first decoding iteration, the subtraction is not necessary for the pre-processing unit. As such, channel data will go directly to the CNU array through the shifter. Only three sub-iterations will take place in the pre-processing unit, less CNU units can be instantiated than the processing unit, to match their throughput. Achieving  $p \times p$  cyclic shifts by  $M \times M$  shifter, where  $M < p$ , is introduced in [8].  $P$  messages and check-node messages are passed from pre-processing unit to processing unit through the frame buffer. The frame buffer size  $K$  is determined by time response of variable voltage supply  $V_{ddl}$  as well as expected decoding time, i.e. decoder throughput. As will be presented later, buffer size of 4 frames is typically needed. The overhead is buffer of size 3 frame since a buffer size of 1 frame is intrinsic for the decoder architecture [7]. Considering the memory-saving nature of the decoder architecture, this overhead is very small.

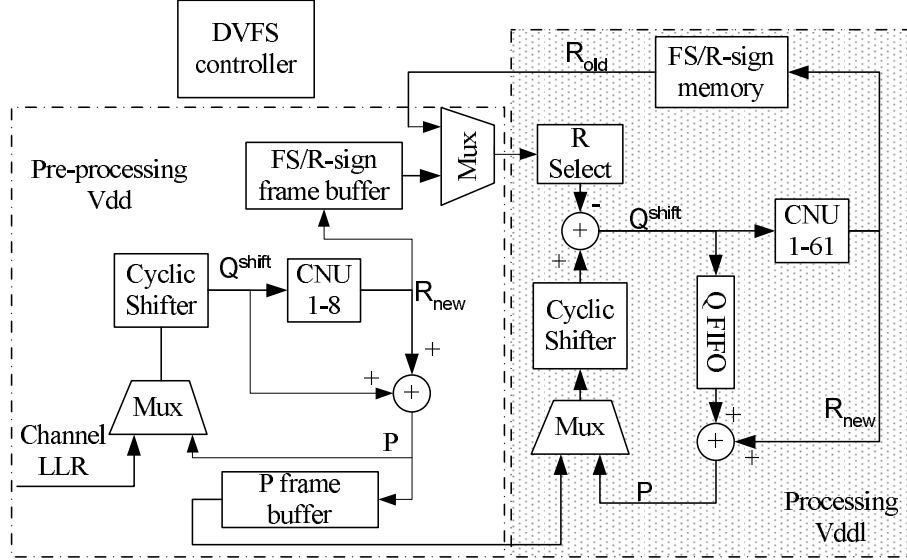


Fig. 8. Proposed decoder architecture for AWGN channel.

Note that other than the frame buffer, the pre-processing unit should not be considered as hardware overhead of the adaptive decoder architecture because it is an inherent part of the decoder and it contributes directly to the final decoder throughput.

The processing unit takes  $P$  sum messages and check-node messages from the pre-processing unit and continue decoding. The difference from pre-processing unit lies in that  $Q$  messages instead of  $P$  message are stored. Computation of  $Q$  messages are carried exactly as in Equation (7).

#### IV. RESULTS AND DISCUSSION

The design of voltage-scaling controller has been simulated using *TSMC0.13 $\mu$ m* technology. Critical path of the decoder is extracted from Synopsys design compiler. With 1.5V voltage supply, the decoder can be clocked as fast as 175MHz, as shown in Figure 9. Extra 5% timing margin has been added to the critical path replica in the controller to accommodate variations.

Area of the DVFS controller and variable supply voltage scheme is  $0.1 \times 0.13\text{mm}^2$ , about 1% of the  $1.1\text{mm}^2$  whole decoder and the power overhead is less than 10mW, which mainly comes from the buck converter. This power consumption is small comparing to the total power dissipation of the decoder at around 200mW. The variable voltage scheme is capable of adjusting the  $V_{DDL}$  voltage at a speed of  $20\text{mV}/\mu\text{s}$ . Similar results has been reported in [17]. For LDPC decoding at  $E_b/N_o$  of 3.7dB, the maximum number of decoding sub-iterations  $C_{[1:4]}$  is choose to be 48, 32, 24 and 19, respectively and the corresponding decoder frequency is chosen to be 165MHz, 110MHz, 82.5MHz and 66MHz, which can be easily derived from a 330MHz system clock. As such, it will yield a nearly equal maximum decoding time for frames in each category. The voltage supply varies from 1.5V to 1V within this frequency range. It takes about 25 $\mu$ s to

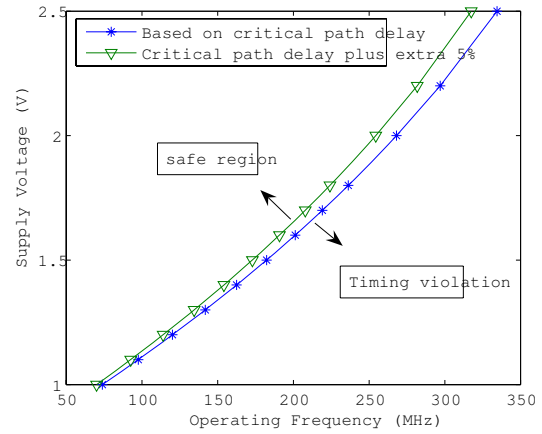


Fig. 9. Voltage supply requirement for different operating frequencies of the processing unit.

scale the voltage up by 0.5V. In the case of  $d_c = 25$ ,  $d_v = 5$  and  $p = 67$ , the voltage controller is able to respond to as much as 200Mbps throughput with a frame-buffer size of 4.

The proposed adaptive decoder architecture saves significant energy consumption. Simulation results is shown in Figure 10, the amount of saving is in comparison with the decoder architecture but without voltage and frequency adjustment. At relatively low SNR level, with  $E_b/N_o$  at 3.4dB for example, up to 30% percent saving in energy consumption is achieved. Note that power consumption is estimated assuming 80% dynamic power and 20% static power consumption. Energy consumption is lowered at high SNR level. At  $E_b/N_o$  of 4.0dB, the saving, though decreases to 21%, is still significant. This decreasing occurs because the reduced average number of decoding iterations. Since the first three sub-iteration takes place in the fixed  $V_{dd}$  voltage domain, as SNR increases,

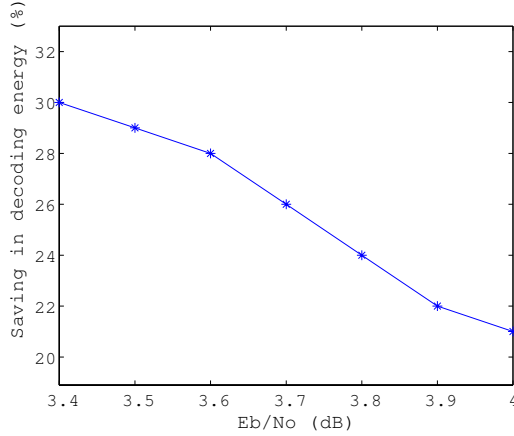


Fig. 10. Saving in decoding energy at different SNR levels.

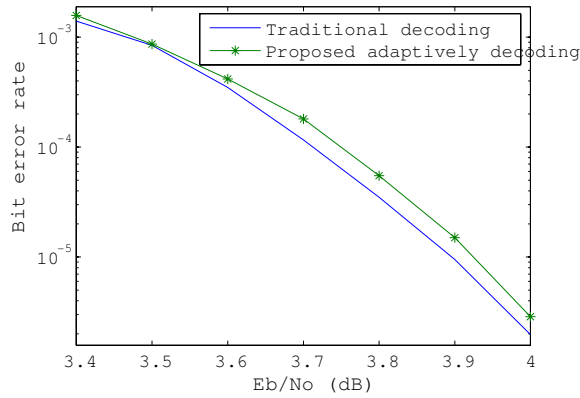


Fig. 11. BER comparison for traditional decoding and proposed adaptive decoding.

decoding energy consumption of the pre-processing block rises relative to the processing unit. Figure 11 shows that the proposed adaptive decoding architecture performs closely with conventional decoding schemes in terms of code performance. As seen, less than 0.05dB BER degradation is observed.

## V. CONCLUSION

We have presented a low-power LDPC decoder design for additive white Gaussian noise (AWGN) channels. Decoding effort for each channel frame is estimated in the early stage of the decoding process. The proposed approach solves the difficult problem of decoding prediction for AWGN channels, and it is also applicable for fading channels. Power overhead of the adaptive decoder control unit mainly stems from the variable voltage supply scheme, and it is small compared with power saved. Up to 30% energy saving in decoding process is achieved with less than 0.05dB coding performance degradation in  $E_b/N_o$ .

## REFERENCES

[1] R. Gallager, "Low-density parity-check codes," *IRE Trans. on Inform. Theory*, vol. 8, pp. 21–28, Jan. 1962.

[2] D. MacKay and R. Neal, "Near shannon limit performance of low density parity check code," *Elec. Letters*, vol. 332, pp. 1645 – 1646, Aug. 1996.

[3] M. Mansour and N. Shanbhag, "High-throughput ldpc decoders," *IEEE Trans. on Very Large Scale Integrated (VLSI) System*, vol. 11, no. 6, pp. 976 – 996, Dec. 2003.

[4] M. Mansour and N. Shanbhag, "A 640-mb/s 2048-bit programmable ldpc decoder chip," *IEEE J. of Solid-State Circuits*, vol. 4, no. 3, pp. 684 – 698, Mar. 2006.

[5] F. Kienle and N. When, "Low complexity stopping criterion for ldpc code decoders," *IEEE 61st Vehicular Technology Conference*, vol. 1, pp. 606 – 609, May 2005.

[6] G. Glikiotis and V. Paliouras, "A low-power termination criterion for iterative ldpc code decoder," *IEEE Workshop on Signal Processing Systems Design and Implementation*, pp. 122 – 127, Nov. 2005.

[7] K. Gunnam, G. Choi, W. Wang, E. Kim, and M. Yeary, "Decoding of quasi-cyclic LDPC codes using on-the-fly computation," *Fortieth Asilomar Conference on Signals, Systems and Computers, 2006. ACSSC 06*, pp. 1192 – 1199, Oct. 2006.

[8] K. Gunnam, W. Wang, G. Choi, and M. Yeary, "Multi-rate layered decoder architecture for block LDPC codes of the IEEE 802.11n wireless standard," *IEEE International Symposium on Circuits and Systems*, pp. 1645–1648, May 2007.

[9] K. Gunnam, G. Choi, and M. B. Yeary, "A parallel layered decoder architecture for array LDPC codes," *Proceedings of the 20th International Conference on VLSI Design*, pp. 738–743, Jan. 2007.

[10] K. Gunnam, G. Choi, M. B. Yeary, S. Yang, and Y. Lee, "Next generation iterative LDPC solutions for magnetic recording storage," *Forty-second Asilomar Conference on Signals, Systems and Computers, 2008*, Oct. 2008.

[11] W. Wang and G. Choi, "Minimum-energy LDPC decoder for real-time mobile application," *Design, Automation and Test in Europe Conference and Exhibition (DATE)*, pp. 1–6, Apr. 2007.

[12] A. Dholakia and S. Olcer, "Rate-compatible low-density parity-check codes for digital subscriber lines," *2004 IEEE International Conference on Communications*, vol. 1, pp. 415 – 419, Jun. 2004.

[13] T. J. Richardson, M. Shokrollahi, and R. Urbanke, "Design of capacity-approaching irregular low-density parity-check codes," *IEEE Trans. on Information Theory*, vol. 47, pp. 619 – 637, Feb. 2001.

[14] F. Kschischang, B. Frey, and H. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. on Information Theory*, vol. 47, pp. 498 – 519, Feb. 2001.

[15] J. Chen and M. Fossorier, "Near optimum universal belief propagation based decoding of low-density-parity-check codes," *IEEE Trans. on Communications*, vol. COM-50, pp. 406 – 414, Mar. 2002.

[16] W. Wang and G. Choi, "Speculative energy scheduling for LDPC decoding," *Proceedings of the 8th International Symposium on Quality Electronic Design*, pp. 79–84, 2007.

[17] T. Kuroda, K. Suzuki, S. Mita, T. Fujita, F. Yamane, and *et al.*, "Variable supply-voltage scheme for low-power high-speed CMOS digital design," *IEEE J. of Solid-State Circuit*, vol. 33, no. 3, pp. 454 – 462, Mar. 1998.

[18] P. Macken, M. Degrauwe, M. van Paemel, and H. Oguey, "A voltage reduction technique for digital systems," *1990 IEEE International Solid-State Circuits Conference, Digest of Technical Papers. 37th ISSCC.*, pp. 238 – 239, Feb. 1990.